بِسْمِ اللهِ الرَّحْمنِ الرَّحِيمِ

Montag, 19. Mai 2025 19:12

Dear colleagues and friends,

As we open the history of AI to understand and use the knowledge of the past in the future. I would like to begin by sharing an inspiring narration from Ali ibn Abi Talib:

"Knowledge is power. Whoever acquires it can rise with it, and whoever lacks it is overpowered." [(1)]

This means that if we want to rise, we need to have knowledge. Yes, sometimes it is hard to get it and to see its purpose. But it is power, and if we don't have it we will get overpowered. We need, for example, this subject "History of AI" to learn from the past and to make new history.

_____

(1) Nahj al-Balagha (The Peak of Eloquence), Saying 660

Lecture Summary

Mittwoch, 21. Mai 2025 15:44

Lecture vs. Book:

Feature	Lecture Perspective	Campbell-Kelly Perspective	Difference Explained
Geographic Focus	Emphasizes the European side of computing and AI history.	Primarily focuses on the American/British perspective.	The lecture adds nuance and highlights developments outside the dominant US narrative presented in the textbook.
Course Scope	Explicitly links history of computing with history of AI.	While covering related topics, the primary focus is the "history of the information machine" (computing).	The lecture foregrounds the AI dimension as a core thread throughout the historical narrative.
Overall Goal	Aims to understand digital culture, reflecting on present/future impact of algorithms.	Presents a history of computing technology and its impact.	The lecture frames the history around the broader socio-cultural concept of digital culture and its implications.

Lecture 2:

A.M. Turing's 1948 research proposal called "Intelligent Machinery". This document is considered very important or "seminal" in the history of AI because it brought together or "connected" various different research ideas that were happening at the time. In this proposal, Turing explored the possibility that a machine could think. He also questioned why people believed that machines could not be intelligent and tried to explain the reasons behind this thinking. He referred to the early electronic machines of the time, like the ADVAC, ADSAC, and the Manchester Baby, in his proposal.

A significant field of study that played a central role during this period was . Cybernetics was not limited to one area but was a combination of various academic subjects. The sources indicate it connected fields such as psychology, which includes the study of language and games like chess; mathematics, particularly formal logic; philosophy; and biology, looking at evolution and populations. The main goal of Cybernetics was to try and copy or imitate the functions of the human brain, or at least those parts of the brain that people focused on. Even though it brought together people with many different interests, Cybernetics was a strong reason or "motivator" for building computers in the early 1950s. People involved in Cybernetics, including unexpected groups like librarians, were interested in both computers and AI. There were different ideas within Cybernetics; some thought human thinking could be described using logic, while others believed comparisons with electronic systems could show what makes human thinking special, including random connections. Norbert Wiener, a key figure, focused his work on feedback loop mechanisms and adaptability, defining intelligent behavior as expected behavior.

The field of Cybernetics also included research into how people could . This shows that the study of Cybernetics and early AI was linked not only to building machines that could potentially think or imitate brain functions but also to understanding and improving education and human interaction with computers. Thinking about how people learn was seen as something that could benefit AI research, and using computers in education was seen as a practical application of AI ideas.

Lecture 3: The Sound of Software

Understanding the history of computers and Artificial Intelligence (AI) involves looking beyond the most common or easily found stories. Often, **history is told by those who were successful or survived**, meaning we hear their side of things most clearly. However, **historians** try to find **other perspectives** and **counterexamples** that might challenge the simple or dominant narratives. To do this, it's crucial to include **details**, as these make historical accounts clearer and more understandable, helping us see the nuances and logic in different people's experiences.

As an example of looking at history from a **non-US perspective**, the story of **Dutch computing** is highlighted. Just like in the United States, people in the Netherlands realized after World War II that computers could be very useful for science, planning for the country, and engineering. However, there were important differences. Unlike the large projects funded by the military and big companies in the US, **Dutch projects were typically smaller** and led by **academic** or **public institutions**. This meant they often faced **tight budgets** and had to constantly **improvise** to get things done. Historian Gerard Alberts famously described this approach as the Dutch engineers "bluffing their way into the computer age," showing their ambition despite limited resources. This contrasts with the popular notion that computers were just invented to make life easier or by computer scientists; such simple stories are based on incomplete information. Looking at stories like the Dutch one helps provide a more nuanced understanding of how computing developed globally.

When we talk about computing, we often use **metaphors**—words that help us understand how machines work by comparing them to other things. Concepts like "language" for programming, "compiler" for translating code, or "memory" for storing information are examples of these metaphors that help define the technology. Computing also involves **sounds**, which were part of the early machines and how they were perceived.

Another topic mentioned is **real-time computing**, which involved computers needing to process information and respond immediately, often necessary for things like military applications (such as guiding a missile). This is discussed as an additional part of the history, adding to the information found in standard textbooks.

The lecture also explores the connection between **Science Fiction (Sci-fi)** and the rise of **AI**. Sci-fi stories have long imagined intelligent machines and what they might mean for humanity. Comparing works like George Orwell's dystopian novel *1984*, which features themes of surveillance and control, with Isaac Asimov's Robot novels, which often explore the rules and ethics of robots interacting with humans, shows different ways people thought about the potential impact of advanced technology. Sci-fi often asks big questions about what it truly means to be **human**, sometimes suggesting that qualities like **feeling** are more defining than just **thinking**. Popular culture, such as the TV series *Star Trek*, also played a role in exposing the public to ideas about computers and AI. While sometimes the connections between Sci-fi ideas and actual AI research were indirect or "flimsy," these stories helped shape public perception and were places where the possibilities of AI were explored. This shows how cultural ideas circulating in stories can be important alongside the technical developments in the history of AI.

Key Content:

- History is often told by the victors or survivors, but historians seek other perspectives. Using details is crucial to make historical stories clear and vivid.
- The Dutch computing story is highlighted as an example of a non-US narrative. Dutch computing responded to post-war needs like the US, but projects were smaller, run by academic/public institutions, had tight budgets, and required improvisation. This contrasts with the larger-scale, military/corporate funded projects often highlighted in US histories. Gerard Alberts described it as "bluffing their way into the computer age".
- Computing involves sounds and the use of metaphors (like 'language,' 'compiler,' 'memory') which define the technology.
- Real-time computing is mentioned as an addition to the textbook's Chapter 7 story.
- The lecture touches upon Science Fiction (Sci-fi) and the rise of AI, comparing Orwell's 1984 with Asimov's Robot novels. Sci-fi explores the redefinition of what makes us human, focusing on feeling instead of just thinking. Star Trek is mentioned as popular exposure to computers/AI.

Memory Mnemonic: Imagine a Dutch dwarf (European story) trying to "bluff" his way into the computer age with only a small budget. He uses metaphors like "programming language" which sound like human talk but are really instructions for the machine, all while trying to dodge the "Big Brother" figure from sci-fi. Remember the dwarf is improvising because he doesn't have the big US funding.

Feature	Lecture Perspective	Campbell-Kelly Perspective	Difference Explained
European Computing	Provides the specific example of Dutch computing, highlighting its smaller scale, academic/public funding, and improvisation ("bluffing").	Campbell-Kelly's Chapter 6 may cover non-US stories (European dwarfs), but the Dutch example is detailed in lecture.	The lecture provides a specific, detailed case study of European computing challenges and approaches that might not be in the textbook.
Real-Time Computing	Explicitly states that discussion on real-time computing is an addition to Campbell-Kelly's Chapter 7 story.	Campbell-Kelly's Chapter 7 covers certain topics (not detailed in sources).	The lecture adds a specific technical topic (real-time computing) not fully covered in the assigned textbook chapter.

Lecture 4: Computing Crisis?

First off, there was a time in the late 1960s and early 1970s when people started talking about a **"computer crisis,"** specifically a **"software crisis"**. This happened because there were problems with the increasing amount of computer code being written. It's interesting to know that sometimes, declaring something a "crisis" isn't just stating a fact; it can be a **strategic move to push a certain agenda**. People might name a crisis to gain more control over the situation or to promote their own goals or ideas.

During this time, different ways of telling computers what to do, called **programming languages,** were being developed. The lecture highlighted two important ones: **ALGOL 60** and **LISP**, comparing them to other languages like COBOL and FORTRAN that a historian named Campbell-Kelly focused on. It's actually tricky to say exactly when a programming language "starts". Does it start when someone first thinks of the idea, when they write down the rules (syntax), when a program is created to translate it for the computer (compiler), or when it's introduced to people as an option?

One of these languages, **COBOL**, was the idea of a person named Grace Hopper. Her main goal was to make the computer code easier for people who weren't technical experts, like business managers and government workers, to read. She even had this bold idea that the computer could understand instructions written in different human languages, like French or German, not just English. This worried her supervisors at first. COBOL programs often started with a diagram showing the steps, called a flowchart.

Another language, **LISP**, was created by John McCarthy and was connected to the ideas of Alonzo Church, who worked on something called lambda-calculus. LISP was designed to be a more **logical language for computers,** especially for dealing with mathematics and checking if mathematical proofs were correct. This idea of using computers to check mathematical proofs was something McCarthy presented in a paper in 1961. This work is considered part of the early history of **Artificial Intelligence**, even if McCarthy didn't always specifically call it "AI" himself. The link between AI and mathematics goes back to people like Bertrand Russell and Alfred Whitehead and their massive work called *Principia Mathematica*, which tried to write mathematics using formal logic. To mathematicians back then, doing calculations and checking proofs started to seem like similar kinds of logical steps. There was even another project called Automath in the Netherlands that also focused on having computers check mathematical proofs.

Computers started influencing how people thought about education in a few ways. There was the question of whether to teach about computers in school subjects (**computers in the curriculum**). There was also the way computing influenced ideas about how people think and learn (**computers in thinking about education**). And finally, there was the idea of actually using computers to help teach students (**computers in the educational process**).

In schools, especially in the 1960s, concepts related to computers started showing up in mathematics classes. Things like **binary arithmetic** (math using only 0s and 1s, like computers do) and **flowcharts** (diagrams showing steps for a computer program) were introduced, specifically because of the growing importance of computers. Programming itself was also taught, often as part of mathematics. In Europe during the 60s and 70s, the language often taught was ALGOL, not BASIC. The reason for teaching programming was seen as necessary in a society increasingly relying on computers, to help people understand that computers only do what humans tell them to do through algorithms. Early students sometimes wrote programs on punched cards and sent them off to a computer center because there weren't computers in every classroom. Interesting examples, like creating a matchmaking program, were used to make abstract programming ideas more understandable.

Mathematics education was also changing more generally, moving away from just teaching students how to solve specific types of problems, and more towards teaching them **how to think and truly understand** concepts. There was a famous, slightly worrying experiment where students who had learned step-by-step methods tried to solve a completely unrelated question by just adding numbers, showing that they could apply steps without actually thinking or understanding the problem.

This focus on making learning steps clear connects to the work of Benjamin Bloom, who in 1956, created a way to categorize **educational objectives** that made them more measurable. This idea of breaking down learning into clear, measurable steps felt similar to how you structure steps in a computer program. Grace Hopper even used the metaphor of **"educating a computer"** for programming, linking the idea of teaching a machine to teaching a person. This thinking influenced approaches like "programmed instruction," where learning materials were designed like programs, guiding students step-by-step through a subject.

During the early to mid-1960s in the Netherlands, ideas about integrating computers into learning began to emerge. By 1966, **vocational training already included learning a simple code**, possibly related to administrative tasks, as seen in a textbook written by mathematician and chess champion Max Euwe. This suggests an early recognition of the need for students to interact with programming concepts. Further recommendations came in a 1968 report from a Dutch government committee that had been established in 1961. This committee strongly advised including **"computer mathematics"** in secondary education, with a specific focus on teaching programming. Interestingly, they felt the particular programming language wasn't the most crucial aspect, as long as it was ALGOL.

The justification given for making computing and algorithms a necessary part of the curriculum was quite significant. It wasn't just about job skills; it was seen as vital for the public to understand how computers actually work. The concern was that in a society increasingly reliant on computers, people might uncritically accept any output from a machine simply because an "authority" presented it. There was a fear of a potential "big brother" scenario where computer results would be accepted without question. To counter this, the committee believed it was essential for the general population to realize that computers don't act on their own. Instead, **people "do things with help of a computer," and these actions are based on algorithms**—step-by-step instructions that humans design in advance to tell the computer how to respond in different situations. Therefore, showing students what happens *before* a problem can be solved by a computer was crucial. Because of this societal need, they felt it was **"inevitable to teach programming"**.

In practice, teaching programming in the 1960s and early 1970s was challenging because computers weren't readily available in every classroom. In the Netherlands, students often programmed using **punched cards**. These cards, which contained the coded instructions, had to be collected and sent off to a central computer center. The students would then receive the results days or even a week later. Despite this indirect access, computers were viewed as valuable tools, particularly for creating

mathematical models. To make abstract programming concepts more concrete and engaging, teachers used examples like programming a **matchmaking algorithm** to arrange blind dates among pupils, which helped students understand the use of things like arrays (structured lists of data). This showed that while rooted in mathematics, the application could be relatable to students' lives.

Access to computing resources varied. Unlike the somewhat indirect access for most students in the Netherlands, some other places offered more direct interaction. For example, in **Czechoslovakia in the early 1970s, special classes were established for students who were particularly good at mathematics**. The idea was that those skilled in math would also be good at programming. These top students in these special classes **actually had direct access to computers**, which was quite special and costly at the time. This contrasted with the Dutch approach where direct access was typically limited to universities with expensive machines.

Connecting the idea of teaching computers to human learning processes was a significant metaphor. **Grace Hopper, a pioneer in computing, famously used the metaphor of "educating a computer" when talking about programming**. This concept tied into psychological theories prevalent at the time, such as **behaviorism**, which viewed learning as a form of conditioning. This psychological perspective led to the development of tools like **Skinner machines** and **programmed instruction manuals**. These learning aids were designed to break down subjects into small, manageable steps, guiding the student through the material incrementally. This step-by-step approach strongly mirrored the structured, algorithmic way you would design steps in a computer program. This method was quite influential in educational thinking during the 1950s and 1960s, although the popularity of programmed instruction materials (like the books) began to decline rapidly around 1970.

Interestingly, **early researchers in Artificial Intelligence were often involved in creating these programmed instruction materials and later, Computer-Assisted Instruction (CAI) systems**. They saw this work in education as a practical application of their AI ideas. For them, developing methods to structure learning and guide students felt related to teaching a machine or simulating intelligence. Another notable project led by key AI figures, Seymour Papert and Marvin Minsky, was the **LOGO project**, which started in 1967. LOGO was designed to introduce programming concepts to **primary school pupils** using a simple command language to control a robot "turtle". This allowed very young children to explore geometric ideas and programming in a playful way, serving as another example of how early AI researchers directly engaged with and influenced the field of education.

Key Content:

- The term "computer crisis" (or "software crisis") was proclaimed in the late 1960s/early 1970s due to emerging problems.
- Naming something a crisis can be a way to push an agenda, aiming for better control or promoting specific goals.
- Programming languages are discussed, specifically ALGOL 60 and LISP, comparing them to Campbell-Kelly's focus on COBOL and FORTRAN. Different interpretations exist for when a language "starts" (conceived, syntax, compiler, presentation).
- COBOL was initiated by Grace Hopper to make code more readable for managers and government institutions. She proposed it could work in any language, initially alarming supervisors.
- LISP, designed by John McCarthy, was intended as a logical language for the computer, enabling mathematics and proof checking. It was influenced by Alonzo Church's lambda calculus and the work of Whitehead and Russell's Principia Mathematica. McCarthy presented a paper on computer programs for checking mathematical proofs in 1961. This work is presented as part of the history of AI, even if McCarthy didn't always explicitly label it as such.
- Computers in education has three components: computers in the curriculum (binary arithmetic, flowcharts, programming), computers in thinking about education (psychology, learning abilities), and computers in the educational process (using computers in class/textbooks).
- Binary arithmetic and flowcharts were introduced in mathematics curricula in the 1960s, motivated by computers.

- Programming was taught, often ALGOL in Europe, not BASIC in the 60s/70s schools. This happened within mathematics curricula.
- Mathematics education began emphasizing learning to think and understanding understanding, rather than just solving equations. A disturbing experiment showed students solving problems without thinking if they could just apply steps.
- Benjamin Bloom's Taxonomy (1956) made educational objectives measurable, linking to the idea of structuring learning like programming.
- In the Netherlands, vocational training included learning a simple code by 1966 (Max Euwe textbook). A 1968 report by a Dutch government committee recommended including "computer mathematics" in secondary education, focusing on programming (ALGOL).
- The justification for teaching computing/algorithms was the need for the public to understand that computers don't act alone, but are tools used by people who design algorithms in advance, to prevent a "big brother" scenario where computer output is accepted uncritically. Teaching programming was seen as inevitable.
- In the absence of computers, students programmed on punched cards sent to computer centers. Computers were seen as tools to make mathematical models. A matchmaking algorithm example demonstrated the use of arrays.
- Czechoslovakia had special classes for gifted math students with direct computer access in the early 1970s, unlike the Netherlands where access was indirect.
- Grace Hopper's metaphor "educating a computer" (programming) connected to psychological ideas like behaviorism and conditioning. This linked to Skinner machines and programmed instruction manuals which broke down learning into steps, mirroring programming. This approach was influential in the 50s/60s but declined by 1970.
- Early AI researchers were involved in programmed instruction/CAI, seeing it as AI in practice.
- The LOGO project (Papert and Minsky, 1967) aimed to introduce programming to primary school pupils using a robot turtle. This was another example of AI people working in education.

Memory Mnemonic: Imagine the software crisis (late 60s/early 70s) as a tangled ball of yarn. People like Hopper (COBOL) wanted to make it neat for managers, using words they understood. McCarthy (LISP), the mathematician, saw the yarn as logic, trying to check proofs like those in Principia Mathematica. Meanwhile, in education, teachers (like Bloom) were trying to measure learning like programming steps (Taxonomy), or make kids "educate" a computer (Hopper's metaphor) using Skinner machines or punched cards because real computers were locked away, except for maybe in Czechoslovakia. The goal? Prevent Big Brother by teaching everyone that humans are the ones pulling the algorithmic strings.

Lecture vs. Book:

Feature	Lecture Perspective	Campbell-Kelly Perspective	Difference Explained
Programming Languages Focus	Focuses on ALGOL 60 and LISP as significant languages emerging in the mid-50s/early 60s, emphasizing their connection to mathematics and logic (LISP) and their use in Europe (ALGOL).	Focuses on COBOL and FORTRAN in the programming chapter.	The lecture provides a different set of early languages, highlighting their specific historical context and connection to scientific/mathematical communities, complementing the textbook's business/scientific focus.
Computers in Education Scope	Presents a multi-faceted view: computers in curriculum, in thinking about education (psychology), and in the educational process. Provides	While the textbook might mention aspects of computers in education, the lecture dedicates	The lecture offers a much deeper and broader exploration of how computers and AI influenced educational theory and

	specific examples like Bloom's Taxonomy, programmed instruction, Dutch/Czechoslovak curricula, and the LOGO project.	significant time and specific examples to this topic.	practice, illustrating the practical application of AI concepts in daily life (even if not labelled AI).
History of AI & Mathematics	Strongly emphasizes the connection between AI and mathematics, highlighting the influence of Principia Mathematica, lambda calculus, and proof checking on AI development (e.g., LISP).	The textbook includes AI as a topic, but the lecture specifically claims that this is the "mathematician's perspective" on AI development.	The lecture provides a particular lens on AI history, focusing on the contributions and perspective of mathematicians involved from the beginning, adding a specific academic dimension to the broader history.

Lecture 5: Dream Machines and Modes of Computing

Campbell-Kelly drew a comparison between the **rise of personal computers** in the late 1970s and early 1980s and the earlier rise of **radio in the 1920s**. His point was that in both cases, it was the **content**—for radio, things to listen to, and for personal computers, **software and games**—that made people want to buy the machines. If there wasn't interesting content, owning the device wouldn't be compelling.

A key development that helped broaden the appeal of computers to a wider audience was the creation of the **Graphical User Interface (GUI)**. This interface used a "**desktop metaphor**" with visual elements like windows, icons, and pointers to make interacting with the computer seem more familiar and less technical to "**ordinary people**". However, this new, more accessible interface was sometimes given a critical nickname by more technical users: the **WIMP interface**. This stood for its main components: **Windows, Icons, Mouse, and Pull-down menus**. The term "WIMP" also carries the meaning of a weak or ineffective person. This suggests that some people familiar with older, more technical ways of interacting with computers (like command lines) might have viewed users who preferred the GUI as less capable or knowledgeable. Despite this potential dismissive attitude, the source notes that these "**wimps**" (the ordinary users drawn in by the GUI) were actually **the basis of the machine's success**.

When looking at the history of computing and its popularization, it's important to be precise with definitions and dates. For instance, the claim that the **Amateur Computer Society (ACS)**, founded in **1966**, was the first personal computer club is considered problematic. This is because **personal computers** as they came to be known, such as the **Altair 8800**, didn't really emerge until **1975 or later**. While people in earlier time-sharing systems might have felt they were using a machine personally, this was different from owning a dedicated personal computer. The ACS was certainly a computer club, but it predated the actual era of personal computers.

The provided information also touches upon early attempts to define and document the field of Artificial Intelligence (AI). **Pamela McCorduck**'s book "**Machines Who Think**," published in **1979**, is mentioned as an early history of AI. Notably, it **starts its history with the ancient Greeks**. This approach of beginning a history very early, sometimes called "**bolding history**" or prehistory, can be seen as a way for a field to **claim antiquity and importance**, suggesting that the core ideas have been pursued for a long time. This psychological tactic helps to make the field seem more significant.

Furthermore, the sources indicate that the area known as **Silicon Valley** was a key location for AI research even before the personal computer boom. Significant **AI labs were present there at universities like Berkeley and Stanford** dating back to the **1950s**.

Games played a significant role in the early history of computing and artificial intelligence, serving as both tools for understanding machines and benchmarks for AI capabilities.

From the early days of computing, **playing with machines**, or **tinkering**, was seen as a way of

appropriating them. Even expensive mainframes were used for simple games like Tic-Tac-Toe, which involved building specialized output devices to allow interaction. This playful interaction helped engineers and users become **familiar with the machines** and could even **stimulate programming**.

A particularly significant connection exists between **chess and AI**. Chess has been called the "**Drosophila of AI**". This metaphor compares chess to the fruit fly (*Drosophila*), which was extremely useful for early genetics research but also potentially **fixated the field** by limiting the scope of study. Similarly, chess provided a clear problem for AI research but might have narrowly focused efforts. As far back as 1912, an **electromechanical machine** was created that could play chess endgames.

In 1952, **Alan Turing wrote a chess algorithm**. Because the computers of the time, like the Manchester baby, lacked sufficient memory, **Turing had to execute the program himself**, essentially playing the role of the machine. The idea of chess-playing computers also appeared in **popular culture**, notably with the AI character **HAL in the 1968 movie *2001: A Space Odyssey***, who is depicted playing chess against a human opponent. This reflects the public association of chess with machine intelligence.

AI research in chess often focused on **Claude Shannon's Type B algorithms**. Type B algorithms use methods like **evaluating positions** and looking ahead several moves (**minimax-algorithm combined with an evaluation function and plausible moves**) to choose the best move, aiming for quicker results. This contrasted with Type A algorithms, which would aim for the machine to learn the rules and strategy itself. **IBM was a significant funder** of this type of chess AI research. Their work culminated in **Deep Blue beating the then-reigning world champion Garry Kasparov in 1997**. This event was seen as a significant moment, suggesting a **shift in what was considered "intelligence"** demonstrated by a machine. Simply being able to play chess was no longer the sole benchmark for machine intelligence.

Beyond chess, other types of games also emerged and connected with computing and AI. **Text-based adventure games**, such as **Oregon Trail** (which appeared around 1971 and was popularized by MECC in 1974), became a popular form of entertainment and even education. These games were often played on **teletypes** connected to distant **time-sharing systems**, where multiple users could access a central computer. The player would read a description of a location or situation printed on the teletype and type commands to interact.

These text-based games, despite their often simple programming, appealed to **AI research in language recognition systems**. Researchers were exploring how to get computers to understand and respond to natural language. An example is **Terry Winograd's SHRDLU** (developed between 1968 and 1970). SHRDLU allowed users to interact with a simulated world of blocks using simple English commands, demonstrating that limited **natural language interaction** was possible within a very **limited and predefined environment**. While the connection between complex AI research and the simple text games was sometimes tenuous, these games provided a popular demonstration that felt like talking to a computer.

Key Content:

- The appeal of computers broadened significantly throughout the 1970s and 80s ("Dream Machines").
- Campbell-Kelly compares the rise of personal computers (late 70s/early 80s) to the rise of radio in the 1920s – content (software, games) drove the sales of the machines.
- The Graphical User Interface (GUI) was key to making computers accessible to "ordinary people," using a "desktop metaphor".
- The GUI was sometimes critically referred to as a WIMP interface (Windows, Icons, Mouse, Pull-down menus). The term "WIMP" also means a weak or ineffective person, suggesting a technical user's possibly dismissive view of non-technical users who preferred the GUI. The success of computers was built on these "wimps".
- The founding date of the Amateur Computer Society (ACS) (1966) is noted as problematic when claimed as the first personal computer club, because personal computers (like the Altair 8800) didn't emerge until 1975 or later. This highlights the importance of precise dating and definition in history.

- Pamela McCorduck's "Machines Who Think" (1979) is mentioned as an early history of AI that starts with the ancient Greeks, showing the field's attempt to claim antiquity and importance. Starting a history very early (bolding history or prehistory) can be a psychological tool to make the field seem more significant.
- Silicon Valley had vibrant AI labs (Berkeley, Stanford) since the 1950s.
- Playing with machines (tinkering) was a way of appropriating them from the early days (e.g., Tic-Tac-Toe on a mainframe). Games helped familiarize people with computers and could stimulate programming.
- Chess has a long-standing connection to AI. It's described as the "Drosophila of AI". Just as the fruit fly (Drosophila) was useful for genetics research but also fixated the field, chess was a valuable problem for AI but could limit research scope.
- Alan Turing wrote a chess algorithm in 1952, executing it by hand. The game appeared in popular culture (HAL in 2001: A Space Odyssey).
- AI chess research focused on Shannon's Type B algorithms (using evaluation functions and lookahead search) for quicker results, rather than Type A (machine learning rules itself).
- IBM funded chess AI research, culminating in Deep Blue beating world champion Kasparov in 1997. This event demonstrated a shift in the boundary of what was considered "intelligence" in machines; simply playing chess was no longer the sole benchmark.
- Text-based adventure games (like Oregon Trail from the late 60s/early 70s) emerged, often educational, played on teletypes via time-sharing.
- These text games appealed to AI research in language recognition, such as Terry Winograd's SHRDLU (late 60s) which allowed natural language interaction within a very limited, predefined block world.

Memory Mnemonic: Imagine computers as "Dream Machines" (70s/80s) that lured people in with a friendly face (the GUI/WIMP interface – picture a shy face on the screen). The tech old-timers thought these new users were "WIMPs" for not using code. These dreamers started playing games; chess became the favorite "fruit fly" (Drosophila) for AI researchers. Deep Blue (1997) finally beat the chess champ, but the AI goalpost moved. Others explored text games, talking to the computer like Winograd's SHRDLU robot in a block world.

Lecture vs. Book:

Feature	Lecture Perspective	Campbell-Kelly Perspective	Difference Explained
Personal Computer Clubs Date	Highlights the 1966 date for the ACS as problematic for claiming the "first personal computer club" because personal computers didn't exist until later (1975+). This emphasizes the need for precise definitions and chronological accuracy.	Campbell-Kelly might mention the ACS (mentioned as mentioned by Campbell-Kelly), but the lecture explicitly points out and explains why the 1966 date for a personal computer club is inaccurate in a historical context.	The lecture provides a specific correction or critique of a detail possibly mentioned in the textbook, demonstrating critical engagement with historical claims and the importance of terminology (what constitutes a "personal computer").
AI in Games Scope	Discusses AI in games beyond just chess, including text-based adventure games and their link to language recognition research (e.g., SHRDLU) and the idea of acting as a human opponent in real-time games. Claims these are other ways AI played a role in games.	While Campbell-Kelly likely covers chess, the lecture explicitly presents text-based games and Winograd's SHRDLU as distinct connections between AI research and game culture.	The lecture broadens the view of how AI intersected with game culture beyond the most prominent example of chess, showing diverse applications of AI concepts in interactive entertainment.

History of AI Starting Point	Notes that Pamela McCorduck's 1979 history starts with ancient Greeks, viewing this as an attempt by the field to claim antiquity and importance. Contrasts this with starting points like "prehistory".	Campbell-Kelly's history of computing may also start early (e.g., abacus, tally sticks). The lecture notes that McCorduck's approach for AI history is similar (ancient Greek gods).	The lecture analyzes why historians of a field choose very early starting points, interpreting it as a strategic move to bolster the field's perceived significance, a point of historical methodology rather than just content.

Lecture 6: Networks and AI

Firstly, it's important to understand that **networks existed before the internet**. Examples mentioned include **time-sharing systems** and **military networks like ARPA and NSF networks**. The **internet** as we know it is described as a **network of networks**, which means it connected these previously separate systems together.

AI research is noted as providing an **extra spur or motivation for development**, including network development.

The lecture also briefly mentions the **continued rise of game culture**, building on earlier discussions about games and computers.

A notable cultural phenomenon that emerged was **hacker culture**. Groups like the **Legion of Doom** in the US and the **Chaos Computer Club** in Europe were part of this culture. They often promoted the **accessibility of computers and networks**, sometimes viewing hackers as **modern-day Robin Hoods**. These groups were frequently **politically engaged**, particularly the Chaos Computer Club, which was part of the squatter culture and actively promoted anarchism in the early 1980s. They had distinct identities centered around being savvy with computers and technology.

When looking at the history of the internet, the sources suggest that the **perspective matters**. A historian like Campbell-Kelly might look at it from the perspective of the computer. However, the sources propose that looking at the history of the internet from the perspective of **mass media** (like radio and television) **might make more sense**. Mass media already had widespread reach and existing networks. This perspective highlights that the development was supported by many existing practices.

The sources also suggest a general point about **rapid technological development**: when developments seem to happen very quickly, it usually implies they are **supported by many different actors and agendas** moving together. This makes the process appear fast because various forces are pushing it forward simultaneously.

Regarding the societal impact of AI, the sources point out that while AI is increasingly deployed in various areas, its **benefits are not always universal**. For instance, in hospitals, AI and computer systems can primarily **serve the interests of those profiting** (like hospital management or healthcare insurance companies) but can actually **hinder the daily practice of nursing**. Nurses are sometimes obstructed in their work by the data they have to provide. As a result, nurses often learn to "**play the system**" or "**cope**," navigating around restrictive AI or computer systems to get their work done effectively.

Finally, educational institutes are described as being somewhat **resilient to adopting new technology**. One reason suggested is that new technology, such as electronic learning environments, sometimes **adds administrative workload to teachers** instead of reducing it. Furthermore, new technology in education can have the effect of **increasing disparity between students and between schools**.

It's interesting to note that the people often called the **"godfathers of AI,"** specifically Joshua Bengio, Geoffrey Hinton, and Yann LeCun, were given the prestigious **Turing Award in 2019**. This award recognized important work they had done during a time commonly referred to as the **"Second AI Winter,"** which is generally understood as covering the 1980s and 1990s. The term "AI

winter" refers to periods when there was less excitement and funding for AI research, sometimes because previous promises hadn't been met. However, the fact that these leading figures received a major award for work done during this supposed "winter" suggests that significant progress in AI, including **statistical ways for computers to understand language** (emerging in IBM research in the late 80s) and **research into neural networks** (hyping up in the 90s), was indeed happening during this time. This challenges the idea that AI development completely stopped or froze during these "winters".

One reason why it might seem like AI research slowed down during certain periods is that AI sometimes **"guises itself"** or appears under different names and in different fields. Instead of being openly called "Artificial Intelligence," related research might be conducted within areas like theoretical physics institutes or be given newer, more fashionable names like "machine learning" or "deep learning" to attract interest and funding. This means that important AI-related work could be ongoing, even if it wasn't always labeled as "AI" in the same way.

An interesting example from the history of technology is the **Postmodernism Generator**, which appeared in 1996. This program was one of the early examples of **text generation**. It worked by using keywords and a specific structure to create texts that looked convincing, particularly in the style of postmodern writing. The sources note that creating convincing-looking postmodern text was "rather easy" because this style often relies on specific keywords and phrases that can be put together. This program was developed in the context of academic debates at the time.

Finally, historian **Pamela McCorduck** is mentioned again, highlighting her role in bringing attention to AI research happening outside of the well-known centers like the United States. Her work, such as the book "The Fifth Generation" from 1983 (potentially co-authored with "Helen McCorduck"), pointed out important efforts in other countries, like **Japan's research into self-driving cars** in the 1970s. She also wrote what might be the first history of AI in her 1979 book "Machines Who Think", showing that AI was considered a significant field even then.

Key Content:

- Networks existed before the internet, such as time-sharing systems and military networks like ARPA and NSF networks. The internet is a network of networks, connecting these previously separate systems.
- AI research provided an extra motivation for network development.
- The lecture briefly mentions the continued rise of game culture.
- Hacker culture emerged, with groups like the Legion of Doom (US) and the Chaos Computer Club (Europe). They promoted accessibility (computers and networks), viewing hackers as modern-day Robin Hoods, and were often politically engaged.
- The lecture contrasts looking at the history of the internet from the perspective of the computer (as Campbell-Kelly might) versus from the perspective of mass media. The latter perspective might make more sense, as mass media (like television) had widespread reach.
- Rapid technological development is often supported by many different actors and agendas.
- AI is increasingly deployed in various areas, but its benefits are not always universal. In hospitals, AI systems can serve the interests of those profiting but hinder the practice of nursing. Nurses often "play the system" to get their work done, navigating around restrictive AI/computer systems.
- Educational institutes are described as resilient to adopting new technology, sometimes because it adds administrative burden to teachers. New technology in education can also increase disparity between students/schools.
- The "godfathers of AI" (Bengio, Hinton, LeCun) received the Turing Award in 2019 for work done during the Second AI Winter (80s/90s), suggesting that important AI progress occurred even during periods of reduced hype. Their work included statistical approaches to language and neural networks.
- AI sometimes "guises itself" in other fields, like theoretical physics, before being recognized as AI.
- The Postmodernism Generator (1996) is mentioned as an early example of text generation, using keywords and structure to create convincing, albeit meaningless, text.

- Carol McCorduck is mentioned again, noting her work highlighting AI research in other countries like Japan (self-driving cars).

Memory Mnemonic: Imagine the internet as a giant spiderweb (network of networks). Before it was connected, different spiders had their own small webs (time-sharing, military nets). Hackers with names like "Legion of Doom" saw the web and wanted everyone to access it freely. Looking at the web, some see it as a computer part, but maybe it's better seen as just another way to send messages, like the old mass media. But beware, sometimes the web's design, even with AI, helps only certain spiders, like in hospitals, where nurses have to "play the system" to work. Even the "godfathers of AI" spun their best web during the cold AI Winter, showing that work continued even when funding was low.

Lecture vs. Book:

Feature	Lecture Perspective	Campbell-Kelly Perspective	Difference Explained
Internet History View	Suggests viewing the history of the internet from the perspective of mass media might be more appropriate and make more sense than solely from the perspective of the computer.	Campbell-Kelly's history of the internet (Chapter 9) is framed from the computer perspective.	The lecture offers an alternative conceptual framework for understanding the historical development of the internet, suggesting that its societal impact aligns more with mass media.
AI in Practice Examples	Provides specific, detailed examples of AI's impact and challenges in real-world settings like hospitals (hindering nurses, requiring "playing the system") and the outsourcing of data tagging for self-driving cars.	Campbell-Kelly discusses AI's impact (e.g., self-driving cars, Uber).	The lecture goes into greater detail about the practical and often problematic daily reality of using AI systems for those working with them (like nurses), highlighting the gap between intended function and actual practice.

Lecture 7: Digital Culture, Digital Divide

One main goal is to help you understand the **rise of digital culture**. Another is to encourage you to **reflect on the direction society is heading**. Ultimately, the course aims for you to become **reflexive** about the role your own field of study plays within this context.

The concept of **digital culture** is a central focus. It's noted that "culture" itself can be seen as somewhat hard to grasp or "effervescent". However, the related idea of a "digital divide" – the split between those who have access to technology and those who don't – is described as a very "digital" concept in the sense that it creates a clear boundary: you either belong or you don't belong.

The lecture strongly emphasizes that simple, straightforward historical stories about how computers came about are often **not entirely true**. History isn't just one simple narrative; it's complex and involves many different **desires, intentions (agendas), and viewpoints**. Because of this, there isn't just *the* single, definitive history, but rather **many different histories** that can be told depending on the perspective you take.

The importance of **stories and storytelling** is a recurring theme. Stories are powerful tools because they help **build community**. However, you are warned to **beware of "bad stories"**. These often involve vague accusations that blame anonymous groups like "the government" or simply refer to "they". True history, it is argued, is ultimately about **people making decisions**. Examples given include **Grace Hopper**, described as "educating a computer," which really meant she was programming it, and the original human "computers" – people, often women, who performed calculations before electronic machines took over these jobs.

To avoid staying in an intellectual "bubble" and to get a fuller understanding, it's considered crucial to actively seek out **oddities, counterexamples, and what some might call "missed opportunities"**. These alternative examples show that the standard narratives aren't the only way things could have happened and highlight different paths and ideas that existed. Several specific examples are mentioned:

- The early **Macy conferences on cybernetics**, starting in 1946, are pointed out as discussions about related topics that happened before the commonly cited 1956 Dartmouth conference on AI.
- The **Logic Theorist** program developed by Simon, Newell, and Shaw in 1956 is highlighted. This program was notable for proving theorems from Russell and Whitehead's complex mathematical work, **Principia Mathematica**. It's also mentioned that this program initially didn't even need a physical computer to run; the people involved performed the steps of the algorithm themselves.
- Another example of early international discussion is the 1951 conference in Paris titled "Les machines à calculer et la pensée humaine" ("Calculating Machines and Human Thought").

To get a richer understanding, it's important to consider various viewpoints. For instance, while a standard history of the internet might focus on technical developments or military needs from a computer science perspective, looking at its rise through the lens of **mass media history** (like radio or television) offers a different view. This perspective can highlight the existing infrastructure provided by telephone networks and the critical role of **content** in attracting users, much like broadcasting did for radio.

Secondly, understanding the history involves examining both **successes and failures**. Examples like the **Digital City** in Amsterdam from 1993 show an early "success" in creating online community and demonstrating the demand for internet access, even leading to modem shortages. On the other hand, projects can "fail" in their original purpose but succeed in unforeseen ways. The sources mention the Chinese bulletin board **CFido**, which existed from 1991 to 1998. While it eventually declined and closed, partly due to government surveillance, this **failure from the perspective of user freedom was a success for government surveillance capabilities**, allowing them to learn how to monitor network technology. This highlights that what might be a failure for one group or agenda can turn into a success for another.

Thirdly, the history shows interesting **connections, sometimes with mutual benefits**, between fields like **AI and education**. Ideas from computing and early AI research influenced educational approaches. For example, concepts such as **programmed instruction**, which involved breaking down learning into small steps and often drew analogies to programming, and even aspects related to structuring learning goals (like **Bloom's Taxonomy**) had ties to AI thinking. In return, education provided environments for AI researchers to experiment with ideas about learning and intelligence. The **Logo project**, developed by key AI figures like Minsky and Papert, is a notable example, where primary school children interacted with a robot turtle using simple programming, providing a practical context for exploring concepts relevant to AI research on learning and problem-solving. The overlap in people involved in early AI and these educational projects underscores this mutual influence.

Finally, the sources stress that history is ultimately about **people**. Rather than blaming anonymous entities like "the government" or vague groups referred to as "they," it's important to remember that individuals make decisions and shape events. Successes, especially in technological fields, are often built upon the **importance of people and their connections**. The example of **Bill Gates** is used to illustrate this point. While his technical and business acumen were vital, his early crucial interactions, particularly the deal with IBM for an operating system, were greatly facilitated by his mother, **Mary Gates**. Mary Gates was a well-connected person who knew influential figures, including the CEO of IBM, demonstrating how personal networks and relationships can play a significant role in major historical developments. History, therefore, is about understanding the actions and relationships of the people involved.

Key Content:

- The lecture revisits the course goals: understanding the rise of digital culture, reflecting on society's direction, and becoming reflexive about the role of your field of study.
- Digital culture is the main focus. It's noted that "culture" is effervescent, while "divide" is a digital concept (belong/don't belong).
- The lecture emphasizes that simple historical narratives about computers are not true; history involves different desires, agendas, and perspectives. There isn't the history, but rather many histories told from various viewpoints.
- The importance of stories and storytelling is stressed. Stories build community.
- Beware of "bad stories" that blame anonymous entities like "the government" or "they"; history is ultimately about people making decisions. Examples include Grace Hopper programming ("educating a computer") or the human "computers" who performed calculations.
- The importance of seeking out oddities, counterexamples, and missed opportunities is crucial to avoid staying in an intellectual "bubble".
- Examples of counterpoints include the early cybernetics conferences (Macy conferences, 1946 onwards) pre-dating the commonly cited 1956 AI conference, and the Logic Theorist (Simon, Newell, Shaw) which proved mathematical theorems without needing a physical computer for execution initially. The 1951 Paris conference "Les machines à calculer et la pensée humaine" is another example of early global discussion. The Logic Theorist proved theorems from Principia Mathematica.
- Looking at history through different perspectives (e.g., mass media for the internet) is important.
- Understanding successes and failures is key. Examples include the Digital City or the Chinese bulletin board CIO. Failures can be successes for others (e.g., government surveillance learning from a failing platform).
- Connections between AI and education had mutual benefits; AI concepts influenced education (programmed instruction, Bloom's Taxonomy), and education provided experimental grounds for AI ideas.
- The importance of people and connections in success is highlighted, using the example of Bill Gates being the son of Mary Gates, who had crucial connections (e.g., to the CEO of IBM).

Exam tips are provided:

- questions are similar to assignments, structure your answers (essay or storyline), explain nouns/expressions, use examples for every statement, consider counterexamples if daring, pick questions you know well, use whitespace, history is about people, not just facts. Each question is worth 2 points. The exam is about reflecting on specific topics, not just memorization.

Memory Mnemonic: Imagine the end of the course journey arriving at "Digital Culture City". The entry gate is the "Digital Divide" – are you in or out? You realize the map you used wasn't the map, but a story. Good stories are about people (like Grace Hopper teaching a computer), not invisible "they". To truly understand the city, you must wander off the main path, find the oddities and counterexamples (like the Logic Theorist proving math without a computer, or early cybernetics meetings pre-1956). Remember that even big successes like Bill Gates got help from connections (his mom, Mary Gates). Your exam is your chance to tell your structured story about these people, ideas, and their examples.

Lecture vs. Book:

Feature	Lecture Perspective	Campbell-Kelly Perspective	Difference Explained
Nature of History	Emphasizes that there is no single "the" history, but multiple histories driven by different agendas and	Presents a coherent, largely chronological narrative of computing history.	The lecture offers a meta-level reflection on the nature of historical writing itself, encouraging critical thinking

	perspectives. Stresses the importance of counterexamples and overlooked events/people.		about dominant narratives and seeking out alternative perspectives and evidence that might not fit neatly into the main story.
AI History Events	Highlights early events and concepts like the Macy Cybernetics conferences (pre-1956) and the Logic Theorist as crucial parts of AI history, predating or existing alongside the commonly cited Dartmouth workshop.	Campbell-Kelly likely focuses on the Dartmouth workshop (1956) as a key founding moment for AI (as is common).	The lecture challenges the standard narrative of AI's origins by emphasizing earlier or parallel developments, demonstrating that AI's roots are broader and deeper than a single founding event.
Role of Individuals	Strongly emphasizes that history is about people making decisions, not just anonymous forces or governments. Uses examples like Grace Hopper, the human computers, and Mary Gates's influence.	While mentioning key figures, the overall narrative can sometimes emphasize technological or institutional developments more broadly.	The lecture provides a human-centered perspective on history, arguing that focusing on individuals and their actions/connections makes the past more relatable and highlights the agency of people in shaping technological development.

Part One: Before the Computer (Chapters 1-3)

Montag, 19. Mai 2025 19:20

Focus: This part sets the stage by showing how information processing was handled before electronic computers and introduces foundational mechanical and conceptual ideas. Questions here focus on early definitions, Babbage's contributions and struggles, Ada Lovelace, and pre-electronic business machines like punched cards. The "First computer" question also ties in here by discussing the difficulty of defining a single "first" machine, implying that precursors are part of the story.

Critical Content:

- **The Human "Computer":** Before electronic machines, a "computer" was an occupation. These were people, often employed for mathematical calculations in fields like astronomy or surveying. Table-making was a significant task, where humans computed mathematical tables (like logarithms or nautical tables).
  - **Division of Labor:** The principle of division of labor, influenced by figures like Adam Smith, was applied to table making by people like de Prony. This meant breaking down large calculations into smaller, repetitive steps that could be done by many human computers working in parallel.
  - **Mnemonic:** Imagine a room full of people with calculators, each doing just one type of math problem on a big table. This "Human Computer Factory" used Division of Labor like an assembly line (Smiths and de Pronys overseeing the process) to churn out calculation Tables.
- **Charles Babbage:**
  - **Difference Engine:** Babbage's motivation was to eliminate errors in tables by using mechanism instead of error-prone human calculation. The core method employed was the method of differences. He also designed it to perform a secondary function: automatically typesetting the results to eliminate errors in printing the tables.
  - **Failure to Build:** Despite significant effort and government funding, Babbage ultimately failed to build his engines. Factors contributing to his failure included the machine's mechanical complexity requiring technology that didn't yet exist, constant redesign (often due to his own new ideas like the Analytical Engine), and political misjudgment (especially proposing the Analytical Engine when funding was for the Difference Engine). The engines became an end in themselves, losing sight of the original goal of making tables.
  - **Mnemonic:** Picture Babbage, a brilliant but difficult inventor, trying to build a clockwork engine (Difference Engine) to catch all the mistakes in math books ("unerring certainty of mechanism"). He kept getting distracted by new ideas (Analytical Engine), running out of money, and the parts were too complicated to make with the tools he had. He was like a genius baker who kept inventing new pastries instead of finishing the cake he was paid to make.
- **Analytical Engine:** This was far more significant than the Difference Engine. It was a new kind of engine capable of performing any calculation that a human could specify. In terms of logical organization, it had the same fundamental parts as the modern electronic computer (input, output, memory, arithmetic, control).
- **Ada Lovelace:** She translated and added extensive notes to an article about the Analytical Engine, providing the most detailed account published in Babbage's lifetime. Babbage recognized her crucial role as his "dear and much admired Interpreter". While later scholarship suggests most technical content and programs were Babbage's, her contribution in explaining the engine's potential was significant. She famously used the metaphor of the engine weaving algebraic patterns.
  - **Mnemonic:** Ada Lovelace was the "Interpreter" of Babbage's Analytical Engine. Imagine her explaining the complex machine by showing how it could weave logic and numbers like a textile loom.
- **Punched-Card Tabulation:** Beyond Babbage, early information processing included mechanical

office machinery and punched-card systems. Punched cards were particularly important for handling large-scale data like the US Census. Key figures associated with early development and adoption include Herman Hollerith, whose machines were used for the 1890 US Census. These systems laid groundwork for companies like IBM.

- **Mnemonic:** Think of the US Census. The data was overwhelming, so Hollerith used punched cards that could be read by machines. This was an early way to automate data processing for big jobs, kind of like a giant, noisy, mechanical spreadsheet.

Part Two: Creating the Computer (Chapters 4-6)

Montag, 19. Mai 2025 19:26

Focus: This part covers the birth of the electronic computer during WWII and the shift from scientific tool to business machine, culminating in IBM's dominance. Questions focus on ENIAC, EDVAC and the stored program concept, the transition to business, and IBM's strategy and key machines like the System/360.

Critical Content:

- **ENIAC:** Developed at the University of Pennsylvania during WWII. It is often called the "first electronic computer". It was a giant, powerful calculator used for tasks like ballistic trajectories.
  - **Mnemonic:** Imagine a massive, noisy machine filling a room, covered in glowing vacuum tubes – that's ENIAC, the first electronic giant born from the war.
- **EDVAC and the Stored Program:** ENIAC's successor, the EDVAC (Electronic Discrete Variable Automatic Computer), became a blueprint for almost all subsequent computers. Its key innovation was the "stored-program computer" concept. This meant that instructions (the program) were stored in the same memory as the data, allowing for greater flexibility and easier reprogramming compared to physically rewiring machines like ENIAC.
  - **John von Neumann:** Played a key role in the logical design of EDVAC. He designated five functional parts: central control, central arithmetic part, memory, input, and output organs. He used a biological metaphor, comparing the computer's structure to the organization of the brain, which is the origin of the term "computer memory".
  - **Mnemonic:** Think of EDVAC as the blueprint for modern computers. Its brain, thanks to von Neumann, could store the instructions right alongside the data, like writing the recipe on the ingredient box. Von Neumann even used a brain metaphor – hence, "computer memory."
- **Shift to Business Processing:** The computer transitioned from primarily a "scientific instrument for mathematical computation" to a "machine for business data processing" in the early industry.
  - **Eckert-Mauchly and UNIVAC:** The Eckert-Mauchly computer business (EMCC) and their UNIVAC machine were key players in this shift, aiming for the commercial market. UNIVAC gained fame for predicting the 1952 US Presidential election outcome.
  - **Computer Manufacturers (Exam B type):** One question mentions three types of firms reconstructing the computer for business. Based on the excerpts, these would likely include:
    - 1. The early pioneers like Eckert-Mauchly (EMCC) who designed machines like UNIVAC.
    - 2. Established business machine manufacturers like IBM, who adapted their existing sales channels and customer relationships to the new technology.
    - 3. New startups or other established electronics firms entering the market.

    These firms reconstructed the purpose and application of the computer, packaging it for data processing, payroll, inventory, etc., rather than just complex scientific math.
  - **Mnemonic:** Eckert and Mauchly built UNIVAC and showed the world (by predicting an election!) that computers weren't just for scientists anymore. They were for Business.
- **IBM's Rise:** IBM, already strong in the business machine market with punched-card systems, initially reacted cautiously to electronic computers. Their existing strengths

included a large customer base, a strong sales force, technical expertise, and a rent-and-refill business model. They strategically entered the commercial market, eventually dominating it. The nickname "Big Blue" refers to the company's size, power ("Big"), and its distinctive color/branding ("Blue"). It was considered "sinister" likely due to their overwhelming market dominance and aggressive business practices.

  - **Mnemonic:** IBM was already the Big Blue king of business machines, like a giant, blue-suited salesman. They watched UNIVAC enter their territory, then used their strength and strategic sales tactics to conquer the new computer market.

- **IBM System/360:** This was a key innovation that helped establish IBM's dominance in the mainframe industry. The core strategy was system compatibility. The System/360 was a range of computers with different price/performance points, but they could all run the same software and use the same peripherals. This allowed customers to upgrade without rewriting all their programs, locking them into the IBM ecosystem.
  - **Mnemonic:** The System/360 was IBM's masterstroke. Think of it as a whole family of computers (a "range") that all spoke the same language (compatibility). This made it easy for companies to grow with IBM, like buying different size shirts from the same reliable tailor.

Part Three: Innovation and Expansion (Chapters 7-9)

Montag, 19. Mai 2025 19:27

Focus: This part covers key developments from the early electronic computers to the cusp of the personal computer era, focusing on software, programming languages, the emergence of computer science, and new modes of interaction like time-sharing. Questions target the "software crisis," early programming methods, FORTRAN, computer science as a field, and interactive computing.

Critical Content:

- **The Software Crisis:** By the late 1950s, software development proved "surprisingly difficult". Reasons included that programming was seen as a "black art" rather than a discipline. Early programmers came from varied backgrounds, were self-taught, and lacked standardized practices. Defining and delineating software specifications precisely was also inherently difficult compared to hardware. There was also the initial incorrect assumption that programming ("coding") was simple, routine work for unskilled labor.
    - **Mnemonic:** Picture programmers in the late 50s pulling their hair out – it was a software crisis. Why? Because they were figuring it out as they went, it was a "black art", and nobody really knew how to train people or describe exactly what software should do. It was chaos!
- **Early Programming:**
    - Early computers were programmed using low-level machine instructions.
    - Tools like Initial Orders (a basic routine loaded automatically) and subroutine libraries were developed to make programming more manageable. Subroutines allowed reusable blocks of code. The Cambridge University EDSAC was a pioneer in using these tools and had an important early programming textbook.
    - **Twofold Translation:** Programming was described as a "twofold series of translations". The first translation is converting a real-world problem's solution into an algorithm (a series of machine steps). The second is translating this algorithm into the specific instructions (code) for a particular computer.
    - **Mnemonic:** Programming started like giving very specific orders to a robot using only simple commands (machine instructions). To make it easier, they developed subroutines (pre-written command lists for common tasks) and used "Initial Orders" to get started. It was a two-step translation: first figuring out the steps (the algorithm), then writing them in robot language (the code).
- **FORTRAN:** Developed by IBM, particularly by John Backus. Backus argued for its development on economic grounds – the cost of programming was becoming higher than the cost of hardware, so improving programmer productivity was essential. FORTRAN (Formula Translation) was a high-level programming language. It gained widespread adoption, especially for scientific applications. Its success was partly because it was the first effective high-level language widely available and standardized.
    - **Mnemonic:** IBM hired Backus because programming was getting too expensive (economic arguments). Backus created FORTRAN – a language closer to human math (Formula Translation). It was like finally giving programmers a calculator instead of an abacus.
- **Emergence of Computer Science:** By the mid-1960s, Computer Science emerged as a distinct academic discipline, separate from mathematics and electrical engineering. University programs aimed to address challenges like the software crisis by providing a more rigorous approach to software development and standardizing education. The ACM (Association for Computing Machinery) played a role in this professionalization and standardization.
    - **Mnemonic:** Software was messy, so universities said, "Let's make this a real subject!" Computer Science split off from math and engineering, trying to make programming more like a science or engineering discipline.
- **New Ways of Computing (Time-Sharing, Interactive):** Beyond traditional batch processing (where jobs were run sequentially without user interaction), ideas like "man-computer

symbiosis" and interactive computing emerged as goals.

- **Time-sharing:** Developed at places like MIT, time-sharing allowed multiple users to share a single computer simultaneously, giving each the illusion of having their own machine. This enabled interactive computing, where users could get immediate responses from the computer.
- **J.C.R. Licklider:** A key figure who advocated for "man-computer symbiosis," arguing that computers should enhance human capabilities and work interactively with people on tasks like looking up information and calculating, rather than just pursuing complex AI.
- **Computer Utility (Exam B type):** The idea of the "computer utility" in the 1960s envisioned computing power being delivered like electricity or water. Time-sharing enabled this, as users could connect via terminals to a central computer and pay for usage. Reasons for its initial success included making computing more accessible and affordable than owning a mainframe. Licklider and others involved in time-sharing and early networking projects were key figures.
- **Mnemonic:** Forget feeding punch cards (batch processing). Licklider dreamed of computers working with humans (man-computer symbiosis). This led to time-sharing, like multiple people plugging into the same giant computer "utility" and feeling like they each had their own interactive machine.

Part Four: Getting Personal (Chapters 10-12)

Montag, 19. Mai 2025 19:27

Focus: This part chronicles the shift from mainframes and minicomputers to personal computers and the rise of the internet. Questions cover the cultural context of the PC, user interfaces, the evolution of the web, and historical visions like Vannevar Bush's memex.

Critical Content:

- **Cultural Context of the PC:** The rise of the personal computer wasn't just technological; it had a cultural context. Groups like the "New Communalists" saw computers as tools for empowerment and personal liberation, contrasting with the centralized control of mainframes. Publications like "The Whole Earth Catalog" (associated with figures like Stewart Brand) featured computers alongside tools for self-sufficiency, promoting this view. Ted Nelson advocated for "computer liberation".
  - **Mnemonic:** The PC wasn't just a smaller computer; it was about freedom! "New Communalists" wanted computing power in their own hands, inspired by sources like "The Whole Earth Catalog", which was like a bible for self-sufficient tech gurus.
- **Graphical User Interfaces (GUIs) and the Mouse:** These were significant in making computers more accessible to non-experts. Instead of typing complex commands (like in command-line interfaces such as MS-DOS), users could interact with visual elements on a screen using a mouse. Key institutions/figures associated with pioneering these concepts include Xerox PARC and figures like Douglas Engelbart.
  - **Mnemonic:** Typing commands was like talking to a computer in a foreign language. The GUI and the mouse made it intuitive – pointing and clicking, like interacting with pictures. Think of Xerox PARC as the lab where they invented this user-friendly face for computers.
- **The Internet and Web Evolution:** The World Wide Web transformed from a "relatively static web" (primarily for browsing information) to an "interactive web" or "Web 2.0" in the early 2000s. This change was accelerated by factors like the rise of powerful search engines, widespread use of smartphones accessing the web, and the infrastructure provided by cloud computing.
  - **Mnemonic:** The early web was like a library you could only look at (static). Then came Web 2.0, making it interactive – you could talk back! Smartphones put the web in everyone's pocket, search engines helped find things, and cloud computing provided the power behind it all.
- **Historical Visions of Knowledge Organization (Wells, Bush):**
  - H.G. Wells envisioned a "World Encyclopedia" – a global system for organizing and accessing human knowledge.
  - Vannevar Bush proposed the "memex" in 1945. It was envisioned as a desktop device for individuals to store their books, records, and communications, and to create associative links between items, allowing users to follow trails of information.
  - **Bush's Memex (Exam B type):** The memex was traditional in being a desk-bound device for personal knowledge. It was contemporary for its time in using microfilm and electrical controls. It was futuristic in its core idea of associative linking and non-linear browsing, anticipating hypertext and the interconnectedness of the internet. Early predictions about the Internet's evolution drew upon their visions of organizing knowledge.
  - **Mnemonic:** Wells dreamt of a giant library for everyone (World Encyclopedia). Bush imagined a personal desk machine, the memex, where you could link ideas together. These were like early dreams of the internet, a machine for navigating knowledge.

Part Five: The Ubiquitous Computer (Chapters 13-15)

Montag, 19. Mai 2025 19:28

Focus: This final part examines the modern era where computers are everywhere and intertwined with global, environmental, legal, and political issues. Questions cover globalization, environmental impacts, intellectual property challenges, governance issues (censorship, privacy, etc.), and surveillance capitalism.

Critical Content:

- **Globalization:** Computing is increasingly global, both in production and consumption. This includes the outsourcing of manufacturing to countries in Asia (e.g., through companies like Foxconn) and the rise of innovation and companies in places like Asia (e.g., Lenovo, Huawei, TSMC). Software and services are also offshored.
  - **Mnemonic:** Computers are everywhere (ubiquitous). They're made all over the world (globalization), especially in places like Asia by companies like Foxconn, not just invented in the US.
- **Environment:** Computing is deeply entangled with environmental concerns throughout its lifecycle (manufacturing, use, disposal). An impact mentioned is the energy consumption of data centers. Manufacturing also creates toxic waste.
  - **Mnemonic:** Your computer might be "in the cloud," but the cloud needs huge buildings full of servers (data centers), which use lots of energy. Making the devices themselves also impacts the environment.
- **Intellectual Property (IP):** Modern computing, especially software, has challenged traditional IP law. In the post-WWII decade, lawyers sought to secure patents for computer programming, which was a new area for IP protection.
  - **Mnemonic:** Software is tricky to protect legally. Lawyers had to figure out how to get patents for programs, creating new battles over digital IP.
- **Governance:** Computing is increasingly entangled with governance in various areas. Two key areas include:
  - **Content Control:** Issues like censorship and content moderation enforced by governments and online platforms.
  - **Privacy and Surveillance:** Increasing concerns over individual privacy and government/corporate surveillance in a networked society. Other areas mentioned include political organization/activism.
  - **Mnemonic:** The ubiquitous computer isn't just a tool; it's entangled with power (governance). Who controls what you see online (censorship, moderation)? Who is watching you (privacy, surveillance)?
- **Surveillance Capitalism:** Defined based on concepts like the rise of the interactive web and platforms. It refers to the economic system where personal data is collected and commodified to predict and modify behavior. The rise of interactive platforms contributes by providing vast amounts of user data.
  - **Mnemonic:** On the interactive web, platforms track everything you do to sell ads or predict what you'll buy. This is Surveillance Capitalism – your data is the currency.